

Feb 4th HW 14

1. In the CAP theorem, why is Partition Tolerance (P) considered mandatory in a real distributed system? Given that P is mandatory, what is the actual trade-off a system designer must make?

- In a real distributed system, Partition Tolerance (P) is considered mandatory because network failures are unavoidable. Messages can be delayed, dropped, or entire nodes can become temporarily unreachable due to hardware failures, network congestion, or data center outages. Since partitions will happen in practice, a system must be able to continue operating despite them.
- Given that P is mandatory, the real trade-off a system designer must make is between Consistency (C) and Availability (A) during a partition:
 - CP systems prioritize consistency and may reject requests when partitions occur.
 - AP systems prioritize availability and allow responses that may be temporarily inconsistent.

2. Explain the difference between linearizability and eventual consistency. Give one real-world system example for each.

- Linearizability is a strong consistency model where every operation appears to take effect atomically at a single point in time, and all clients observe operations in the same order. Once a write completes, all subsequent reads must see that write.
 - Example: A distributed lock service (e.g., a coordination service) where correctness depends on all clients seeing the same state immediately.
- Eventual consistency guarantees that if no new updates occur, all replicas will eventually converge to the same value, but intermediate reads may see stale data.
 - Example: DNS or shopping cart systems in large-scale e-commerce platforms.

3. What is a vector clock? How does it determine whether two events are causally related or concurrent? Why can't we simply use physical timestamps instead?

- A vector clock is a mechanism for tracking causality in distributed systems. Each node maintains a vector of counters—one per node in the system—and increments its own counter for each event.
- To determine relationships:
 - If all entries in vector A are $\leq$ vector B and at least one is strictly smaller, then A happened before B.
 - If neither vector dominates the other, the events are concurrent.
- Physical timestamps cannot be reliably used because clocks across machines can drift, skew, or jump due to synchronization errors, leading to incorrect causal ordering.

4. In single-leader replication, what is the split-brain problem during failover, and why is it dangerous?

- The split-brain problem occurs during failover when multiple nodes believe they are the leader at the same time, usually due to network partitions.
- This is dangerous because:
 - Multiple leaders may accept writes independently.
 - Data divergence and irreversible inconsistency can occur.
 - Reconciliation after recovery is complex or impossible.
- Preventing split-brain is critical for maintaining data correctness.

5. Describe the quorum condition ($W + R > N$) in leaderless (Dynamo-style) replication. What happens if this condition is not satisfied?

- In leaderless (Dynamo-style) replication:

- N = total number of replicas
- W = number of replicas required to acknowledge a write
- R = number of replicas queried for a read
- The condition $W + R > N$ ensures that at least one replica involved in the read has the latest write.
- If this condition is not satisfied:
 - Reads may miss recent writes.
 - Strong consistency is not guaranteed.
 - The system may return stale or conflicting data.

6. Compare Two-Phase Commit (2PC) and the Saga pattern for distributed transactions. What is the main weakness of 2PC that Saga avoids, and what guarantee does Saga sacrifice in return?

- Two-Phase Commit (2PC) ensures atomicity across distributed transactions by coordinating all participants, but it has a major weakness: blocking. If the coordinator fails, participants may remain locked indefinitely.
- The Saga pattern avoids blocking by breaking a transaction into a sequence of local transactions with compensating actions for rollback.
 - Weakness avoided: Blocking and single point of failure.
 - Guarantee sacrificed: Atomicity — Saga provides eventual consistency instead of all-or-nothing guarantees.

7. In the Raft consensus algorithm, why does a candidate need its log to be at least as up-to-date as the majority of the cluster in order to win an election? What safety property does this guarantee?

- In Raft, a candidate must have a log that is at least as up-to-date as the majority of the cluster to win an election. This prevents a node with stale data from becoming the leader.
- This guarantees the Leader Completeness property, ensuring that once a log entry is committed, it will not be lost and will appear in the logs of all future leaders.

8. What is a CRDT (Conflict-free Replicated Data Type)? Explain how a G-Counter works and why it always converges without coordination.

- A CRDT (Conflict-free Replicated Data Type) is a data structure designed so that replicas can be updated independently and still converge to the same state without coordination.
- A G-Counter (Grow-only Counter):
 - Each replica maintains its own counter.
 - Counters only increment.
 - Merging replicas takes the element-wise maximum.
- Because increments are monotonic and merges are associative, commutative, and idempotent, the system always converges.

9. Explain the Circuit Breaker pattern. What are its three states, and how does it prevent cascading failures in a microservices architecture?

- The Circuit Breaker pattern prevents cascading failures in microservices by stopping requests to a failing service.
- Its three states are:
 - Closed – Requests flow normally.
 - Open – Requests are blocked immediately after failures exceed a threshold.
 - Half-Open – A limited number of test requests are allowed to check recovery.
- By failing fast and reducing load on unhealthy services, the circuit breaker protects the overall system.

10. What is the difference between range-based partitioning and hash-based partitioning? What problem does consistent hashing solve compared to simple hash-mod-N partitioning?

- Range-based partitioning assigns data based on ordered key ranges:
 - Supports range queries efficiently.
 - Can suffer from hot spots if access is skewed.
- Hash-based partitioning distributes data using a hash function:
 - Ensures even data distribution.
 - Makes range queries inefficient.
- Consistent hashing solves the major problem of simple hash-mod-N partitioning: massive data reshuffling when nodes are added or removed. With consistent hashing, only a small portion of keys need to move, improving scalability and stability.